

AXI Register Slice v2.1

LogiCORE IP Product Guide

Vivado Design Suite

PG373 (v2.1) November 2, 2022

Xilinx is creating an environment where employees, customers, and partners feel welcome and included. To that end, we're removing non-inclusive language from our products and related collateral. We've launched an internal initiative to remove language that could exclude people or reinforce historical biases, including terms embedded in our software and IPs. You may still find examples of non-inclusive language in our older products as we work to make these changes and align with evolving industry standards. Follow this [link](#) for more information.



Table of Contents

Chapter 1: Introduction.....	4
Features.....	4
IP Facts.....	5
Chapter 2: Overview.....	6
Navigating Content by Design Process.....	6
Applications.....	6
Unsupported Features.....	6
Licensing and Ordering.....	7
Chapter 3: Product Specification.....	8
Standards.....	8
Performance.....	8
Resource Use.....	9
Port Descriptions.....	9
AXI Register Slice Parameters.....	13
Multi-SLR Pipelining Parameters.....	14
Advanced Settings Parameters.....	15
AXI Interface Parameters.....	19
Chapter 4: Designing with the Core.....	20
Modes of Operation.....	20
General Design Guidelines.....	29
Clocking.....	30
Resets.....	30
Chapter 5: Design Flow Steps.....	31
Customizing and Generating the Core.....	31
Constraining the Core.....	42
Simulation.....	44
Synthesis and Implementation.....	44

Chapter 6: Example Design.....	47
Appendix A: Upgrading.....	48
Appendix B: Debugging.....	49
Finding Help on Xilinx.com.....	49
Debug Tools.....	50
Appendix C: Additional Resources and Legal Notices.....	52
Xilinx Resources.....	52
Documentation Navigator and Design Hubs.....	52
References.....	52
Revision History.....	53
Please Read: Important Legal Notices.....	53

Introduction

The AXI Register Slice core connects one AXI memory-mapped master to one AXI memory-mapped slave through a set of pipeline registers, typically to break a critical timing path. Several configuration options are available to best suit the nature of the pathway being pipelined.

Features

- Individually configurable for each of the five AXI channels.
- Facilitates timing closure by trading-off frequency versus latency.
- One latency cycle per register-slice by default.
- Able to propagate AXI traffic with no loss in data throughput (without bubble cycles) under all AXI handshake conditions.
- Optional pipelining to cross super logic region (SLR) in stacked silicon interconnect (SSI) devices. See *Large FPGA Methodology Guide: Including Stacked Silicon Interconnect (SSI) Technology* ([UG872](#)) for more information.

IP Facts

LogiCORE™ IP Facts Table	
Core Specifics	
Supported Device Family ¹	Versal ACAP, UltraScale+, UltraScale, Zynq®-7000 SoC, 7 series
Supported User Interfaces	AXI4, AXI4-Lite, AXI3
Resources	Performance and Resource Use web page
Provided with Core	
Design Files	Verilog and VHDL
Example Design	Not Provided
Test Bench	Not Provided
Constraints File	Xilinx Design Constraints (XDC)
Simulation Model	Not Provided
Supported S/W Driver	N/A
Tested Design Flows ²	
Design Entry	Vivado® Design Suite
Simulation	For supported simulators, see the Xilinx Design Tools: Release Notes Guide .
Synthesis	Vivado Synthesis
Support	
Release Notes and Known Issues	N/A
All Vivado IP Change Logs	Master Vivado IP Change Logs: 72775
Xilinx Support web page	

Notes:

- For a complete list of supported devices, see the Vivado® IP catalog.
- For the supported versions of third-party tools, see the [Xilinx Design Tools: Release Notes Guide](#).

Overview

Navigating Content by Design Process

Xilinx® documentation is organized around a set of standard design processes to help you find relevant content for your current development task. All Versal® ACAP design process [Design Hubs](#) and the [Design Flow Assistant](#) materials can be found on the Xilinx.com website. This document covers the following design processes:

- **Hardware, IP, and Platform Development:** Creating the PL IP blocks for the hardware platform, creating PL kernels, functional simulation, and evaluating the Vivado® timing, resource use, and power closure. Also involves developing the hardware platform for system integration. Topics in this document that apply to this design process include:
 - [Port Descriptions](#)
 - [Clocking](#)
 - [Resets](#)
 - [Customizing and Generating the Core](#)
 - [Chapter 6: Example Design](#)
-

Applications

The AXI Register Slice core is used in applications that uses AXI memory-mapped pathways.

Unsupported Features

For information on pipelining AXI4-Stream pathways, see *AXI4-Stream Infrastructure IP Suite LogiCORE IP Product Guide* ([PG085](#)).

Licensing and Ordering

This Xilinx® LogiCORE™ IP module is provided at no additional cost with the Xilinx Vivado® Design Suite under the terms of the [Xilinx End User License](#).

Information about other Xilinx® LogiCORE™ IP modules is available at the [Xilinx Intellectual Property](#) page. For information about pricing and availability of other Xilinx LogiCORE IP modules and tools, contact your [local Xilinx sales representative](#).

Product Specification

Standards

This core adheres to the following standards:

- AXI4
 - AXI4-Lite
 - AXI3
-

Performance

For full details about performance and resource use, visit the [Performance and Resource Use web page](#).

Latency

- **FULLY_REGISTERED Register Slices (each applicable channel):** One latency cycle with no bubble cycles (best-case 100% channel bandwidth).
- **LIGHT_WEIGHT Register Slices (each applicable channel):** One latency cycle with one bubble cycle (best-case 50% channel bandwidth), which is appropriate for AW, AR, and B channel transfers, and all transfers involving AXI4-Lite endpoints.
- **SI Reg or MI Reg:** One latency cycle, no bubble cycles.
- **SLR Crossing Mode and SLR TDM Crossing Mode:** Three latency cycles (of `ac1k`), no bubble cycles.
- **Multi SLR Crossing:** Overall latency varies between 1 and 17 cycles depending on the number of SLR boundaries crossed and the number of pipeline stages configured within each SLR region.

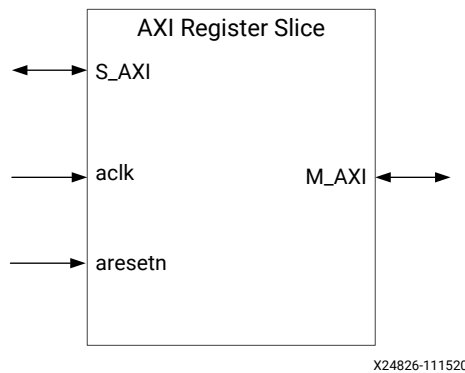
Resource Use

For full details about performance and resource use, visit the [Performance and Resource Use web page](#).

Port Descriptions

The core interfaces are shown in the following figure. For more specific port information, see the following sections.

Figure 1: Core Ports



Slave I/O Signals

The following table lists the Slave Interface signals for the core.

Table 1: Slave I/O Signals

Signal Name	I/O	Default	Width	Description (Range)
s_axi_awid	I	AXI3, AXI4: 0 AXI4-Lite: d/c	ID_WIDTH	Write Address Channel Transaction ID
s_axi_awaddr	I	REQ	ADDR_WIDTH	Write Address Channel Address
s_axi_awlen	I	AXI3, AXI4: 0 AXI4-Lite: d/c	AXI4: 8 AXI3: 4	Write Address Channel Burst Length (0–255)
s_axi_awsz	I	AXI3, AXI4: REQ AXI4-Lite: d/c	3	Write Address Channel Transfer Size Code (0–7)
s_axi_awburst	I	AXI3, AXI4: REQ AXI4-Lite: d/c	2	Write Address Channel Burst Type Code (0–2)
s_axi_awlock	I	AXI3, AXI4: 0 AXI4-Lite: d/c	AXI4: 1 AXI3: 2	Write Address Channel Atomic Access Type (0, 1)

Table 1: Slave I/O Signals (cont'd)

Signal Name	I/O	Default	Width	Description (Range)
s_axi_awcache	I	AXI3, AXI4: 0 AXI4-Lite: d/c	4	Write Address Channel Cache Characteristics
s_axi_awprot	I	0b000	3	Write Address Channel Protection Bits
s_axi_awqos	I	AXI4: 0 AXI4-Lite: d/c	4	AXI4 Write Address Channel Quality of Service
s_axi_awregion	I	AXI4: 0; AXI3, AXI4-Lite: d/c	4	AXI4 Write Address Channel Address Region Index
s_axi_awuser	I	AXI3, AXI4: 0 AXI4-Lite: d/c	AWUSER_WIDTH	User-defined AW Channel Signals
s_axi_awvalid	I	REQ	1	Write Address Channel Valid
s_axi_awready	O		1	Write Address Channel Ready
s_axi_wid	I	AXI3: 0 AXI4, AXI4-Lite: d/c	ID_WIDTH	Write Data Channel Transaction ID for AXI3 Masters
s_axi_wdata	I	REQ	Data Width Converter: S_AXI_DATA_WIDTH; Others: DATA_WIDTH	Write Data Channel Data
s_axi_wstrb	I	All ones	Data Width Converter: S_AXI_DATA_WIDTH/8; Others: DATA_WIDTH/8	Write Data Channel Byte Strobes
s_axi_wlast	I	AXI3, AXI4: 0 AXI4-Lite: d/c	1	Write Data Channel Last Data Beat
s_axi_wuser	I	AXI3, AXI4: 0 AXI4-Lite: d/c	WUSER_WIDTH	User-defined W Channel Signals
s_axi_wvalid	I	REQ	1	Write Data Channel Valid
s_axi_wready	O		1	Write Data Channel Ready
s_axi_bid	O		ID_WIDTH	Write Response Channel Transaction ID
s_axi_bresp	O		2	Write Response Channel Response Code (0–3)
s_axi_buser	O		BUSER_WIDTH	User-defined B Channel Signals
s_axi_bvalid	O		1	Write Response Channel Valid
s_axi_bready	I	REQ	1	Write Response Channel Ready
s_axi_arid	I	AXI3, AXI4: 0 AXI4-Lite: d/c	ID_WIDTH	Read Address Channel Transaction ID
s_axi_araddr	I	REQ	ADDR_WIDTH	Read Address Channel Address
s_axi_arlen	I	AXI3, AXI4: 0 AXI4-Lite: d/c	AXI4: 8 AXI3: 4	Read Address Channel Burst Length code (0–255)
s_axi_arsize	I	AXI3, AXI4: REQ AXI4-Lite: d/c	3	Read Address Channel Transfer Size code (0–7)
s_axi_arburst	I	AXI3, AXI4: REQ AXI4-Lite: d/c	2	Read Address Channel Burst Type (0–2)
s_axi_arlock	I	AXI3, AXI4: 0 AXI4-Lite: d/c	AXI4: 1 AXI3: 2	Read Address Channel Atomic Access Type (0, 1)

Table 1: Slave I/O Signals (cont'd)

Signal Name	I/O	Default	Width	Description (Range)
s_axi_arcache	I	AXI3, AXI4: 0 AXI4-Lite: d/c	4	Read Address Channel Cache Characteristics
s_axi_arprot	I	0b000	3	Read Address Channel Protection Bits
s_axi_arregion	I	AXI4: 0; AXI3, AXI4-Lite: d/c	4	AXI4 Read Address Channel address region index
s_axi_arqos	I	AXI4: 0 AXI4-Lite: d/c	4	AXI4 Read Address Channel Quality of Service
s_axi_aruser	I	AXI3, AXI4: 0 AXI4-Lite: d/c	ARUSER_WIDTH	User-defined AR Channel Signals
s_axi_arvalid	I	REQ	1	Read Address Channel Valid
s_axi_arready	O		1	Read Address Channel Ready
s_axi_rid	O		ID_WIDTH	Read Data Channel Transaction ID
s_axi_rdata	O		Data Width Converter: S_AXI_DATA_WIDTH; Others: DATA_WIDTH	Read Data Channel Data
s_axi_rresp	O		2	Read Data Channel Response Code (0–3)
s_axi_rlast	O		1	Read Data Channel Last Data Beat
s_axi_ruser	O		RUSER_WIDTH	User-defined R Channel Signals
s_axi_rvalid	O		1	Read Data Channel Valid
s_axi_rready	I	REQ	1	Read Data Channel Ready

Master I/O Signals

The following table lists the Master Interface signals for the core.

Table 2: Master I/O Signals

Signal Name	I/O	Default	Width	Description (Range)
m_axi_awid	O		ID_WIDTH	Write Address Channel Transaction ID
m_axi_awaddr	O		ADDR_WIDTH	Write Address Channel Address
m_axi_awlen	O		AXI4: 8 AXI3: 4	Write Address Channel Burst Length Code (0–255)
m_axi_awsz	O		3	Write Address Channel Transfer Size Code (0–7)
m_axi_awburst	O		2	Write Address Channel Burst Type (0–2)
m_axi_awlock	O		AXI4: 1 AXI3: 2	Write Address Channel Atomic Access Type (0, 1)
m_axi_awcache	O		4	Write Address Channel Cache Characteristics

Table 2: Master I/O Signals (cont'd)

Signal Name	I/O	Default	Width	Description (Range)
m_axi_awprot	O		3	Write Address Channel Protection Bits
m_axi_awregion	O		4	AXI4 Write Address Channel Address Region Index
m_axi_awqos	O		4	Write Address Channel Quality of Service
m_axi_awuser	O		AWUSER_WIDTH	User-defined AW Channel Signals
m_axi_awvalid	O		1	Write Address Channel Valid
m_axi_awready	I	REQ	1	Write Address Channel Ready
m_axi_wid	O		ID_WIDTH	Write Data Channel Transaction ID for AXI3 Slaves
m_axi_wdata	O		Data Width Converter: M_AXI_DATA_WIDTH; Others: DATA_WIDTH	Write Data Channel Data
m_axi_wstrb	O		Data Width Converter: M_AXI_DATA_WIDTH/8; Others: DATA_WIDTH/8	Write Data Channel Data Byte Strokes
m_axi_wlast	O		1	Write Data Channel Last Data Beat
m_axi_wuser	O		WUSER_WIDTH	User-defined W Channel Signals
m_axi_wvalid	O		1	Write Data Channel Valid
m_axi_wready	I	REQ	1	Write Data Channel Ready
m_axi_bid	I	AXI3, AXI4: REQ AXI4-Lite: d/c	ID_WIDTH	Write Response Channel Transaction ID.
m_axi_bresp	I	0b00	2	Write Response Channel Response Code (0-3)
m_axi_buser	I	AXI3, AXI4: 0 AXI4-Lite: d/c	BUSER_WIDTH	User-defined B Channel Signals
m_axi_bvalid	I	REQ	1	Write Response Channel Valid
m_axi_bready	O		1	Write Response Channel Ready
m_axi_arid	O		ID_WIDTH	Read Address Channel Transaction ID
m_axi_araddr	O		ADDR_WIDTH	Read Address Channel Address
m_axi_arlen	O		AXI4: 8 AXI3: 4	Read Address Channel Burst Length Code (0-255)
m_axi_arsize	O		3	Read Address Channel Transfer Size Code (0-7)
m_axi_arburst	O		2	Read Address Channel Burst Type (0-2)
m_axi_arlock	O		AXI4: 1 AXI3: 2	Read Address Channel Atomic Access Type (0,1)
m_axi_arcache	O		4	Read Address Channel Cache Characteristics
m_axi_arprot	O		3	Read Address Channel Protection Bits.

Table 2: Master I/O Signals (cont'd)

Signal Name	I/O	Default	Width	Description (Range)
m_axi_arregion	O		4	AXI4 Read Address Channel Address Region Index
m_axi_arqos	O		4	AXI4 Read Address Channel Quality of Service
m_axi_aruser	O		ARUSER_WIDTH	User-defined AR Channel Signals
m_axi_arvalid	O		1	Read Address Channel Valid
m_axi_arready	I	REQ	1	Read Address Channel Ready
m_axi_rid	I	AXI3, AXI4: REQ AXI4-Lite: d/c	ID_WIDTH	Read Data Channel Transaction ID
m_axi_rdata	I	REQ	Data Width Converter: M_AXI_DATA_WIDTH; Others: DATA_WIDTH	Read Data Channel Data
m_axi_rresp	I	0b00	2	Read Data Channel Response Code (0-3)
m_axi_rlast	I	AXI3, AXI4: REQ AXI4-Lite: d/c	1	Read Data Channel Last Data Beat
m_axi_ruser	I	AXI3, AXI4: 0 AXI4-Lite: d/c	RUSER_WIDTH	User-defined R Channel Signals
m_axi_rvalid	I	REQ	1	Read Data Channel Valid
m_axi_rready	O		1	Read Data Channel Ready

Global Port Signals

The following table lists the global port signals for the core.

Table 3: Global Port Signals

Signal Name	I/O	Default	Width	Description (Range)
aclk	I	REQ	1	Clock input.
aresetn	I	REQ	1	Global Reset (active-Low).
aclk2x	I	REQ	1	This auxiliary clock input is only enabled when one or more AXI channels are configured in SLR TDM Crossing mode. The input must be exactly twice the frequency of aclk and should be generated from the same clock source with zero phase shift.

AXI Register Slice Parameters

The following table lists the parameters specific to the AXI Register Slice core.

Table 4: Register Mode Selection Parameters

Parameter Name	Default Value	Config GUI Selections ^{1, 2}	Description
REG_AW	Light	(Bypass, Full, Light, SI_Reg, SLR Crossing, SLR TDM Crossing, Multi SLR Crossing)	AW-channel mode of operation
REG_W	For AXI4 or AXI3: Full For AXI4-Lite: Light	(Bypass, Full, Light, SI_Reg, SLR Crossing, SLR TDM Crossing, Multi SLR Crossing)	W-channel mode of operation
REG_B	Light	(Bypass, Full, Light, MI_Reg, SLR Crossing, SLR TDM Crossing, Multi SLR Crossing)	B-channel mode of operation
REG_AR	Light	(Bypass, Full, Light, SI_Reg, SLR Crossing, SLR TDM Crossing, Multi SLR Crossing)	AR-channel mode of operation
REG_R	For AXI4 or AXI3: Full For AXI4-Lite: Light	(Bypass, Full, Light, MI_Reg, SLR Crossing, SLR TDM Crossing, Multi SLR Crossing)	R-channel mode of operation

Notes:

- Additional legacy modes: "Forward", "Reverse", "Inputs", "SLR Crossing".
- Integer values of REG_* user parameters:
 - 0 = Bypass
 - 1 = Full
 - 2 = Forward
 - 3 = Reverse
 - 6 = Inputs
 - 7 = Light
 - 9 = SI_Reg (for REG_AW, REG_W and REG_AR), or MI_Reg (for REG_B and REG_R)
 - 10 = SLR Crossing
 - 11 = SLR TDM Crossing
 - 15 = Multi SLR Crossing

Multi-SLR Pipelining Parameters

The following table lists the parameters specific to channels of the AXI Register Slice configured in Multi-SLR Crossing mode.

Table 5: Multi-SLR Pipelining Parameters

Vivado IDE Parameter Name	User Parameter Name	Range	Default Value	Description
Number of SLR Crossings	NUM_SLR_CROSSINGS	0-3	0	Number of SLR-crossings between SI and MI (0 means SI and MI are within the same SLR)

Table 5: Multi-SLR Pipelining Parameters (cont'd)

Vivado IDE Parameter Name	User Parameter Name	Range	Default Value	Description
AW-channel Pipeline Stages within Master-side SLR, W-channel Pipeline Stages within Master-side SLR, B-channel Pipeline Stages within Master-side SLR, AR-channel Pipeline Stages within Master-side SLR, R-channel Pipeline Stages within Master-side SLR	PIPELINES_MASTER_AW, PIPELINES_MASTER_W, PIPELINES_MASTER_B, PIPELINES_MASTER_AR, PIPELINES_MASTER_R	0-4	0	Additional pipeline flops between S_AXI interface and SLR boundary flops (or M_AXI interface if NUM_SLR_CROSSINGS = 0)
AW-channel Pipeline Stages within Slave-side SLR, W-channel Pipeline Stages within Slave-side SLR, B-channel Pipeline Stages within Slave-side SLR, AR-channel Pipeline Stages within Slave-side SLR, R-channel Pipeline Stages within Slave-side SLR	PIPELINES_SLAVE_AW, PIPELINES_SLAVE_W, PIPELINES_SLAVE_B, PIPELINES_SLAVE_AR, PIPELINES_SLAVE_R	0-4	0	Additional pipeline flops between SLR boundary flops and M_AXI interface (applicable when NUM_SLR_CROSSINGS > 0)
AW-channel Pipeline Stages within Middle SLR, W-channel Pipeline Stages within Middle SLR, B-channel Pipeline Stages within Middle SLR, AR-channel Pipeline Stages within Middle SLR, R-channel Pipeline Stages within Middle SLR	PIPELINES_MIDDLE_AW, PIPELINES_MIDDLE_W, PIPELINES_MIDDLE_B, PIPELINES_MIDDLE_AR, PIPELINES_MIDDLE_R	0-4	0	Additional pipeline flops between upper and lower SLR boundary flops in each intermediate SLR between the SI SLR and MI SLR (applicable when NUM_SLR_CROSSINGS>1)

Advanced Settings Parameters

The parameters in the following table represent the metadata on the SI and MI AXI interfaces of the AXI Register Slice IP. By default, all parameters are in the Auto mode.

When in the Auto mode, each parameter value is set automatically during BD Validation, according to propagation of AXI metadata from connected IP; then the GUI indicates the resulting values. Typically, metadata values originate at a master or slave endpoint AXI interface, propagate through the register slice, and are consumed by the opposite slave or master endpoint interface at the other end of the pathway. The original metadata values are typically determined during the packaging of the endpoint IP, where they can be set to constants, derived, user-customized, or unspecified (default).

When set in Manual mode, you can force the metadata values to be applied to both the SI and MI interfaces of the AXI Register Slice IP, and potentially propagated to the endpoint IP interface consuming the metadata. This can be useful when the original packaging of the source endpoint IP does not set the metadata to a value suitable for the system design.

Table 6: Advanced Settings Parameters

Vivado IDE Parameter Name	User Parameter Name	Range	Description
WUSER BITS PER BYTE, RUSER BITS PER BYTE	WUSER_BITS_PER_BYTE, RUSER_BITS_PER_BYTE	0-32	0: W/RUSER signal does not scale with DATA_WIDTH (not suitable for width conversion); >0: During width conversion, each field of n bits within W/RUSER travels with corresponding W/RDATA byte lane.
SUPPORTS NARROW BURST	SUPPORTS_NARROW_BURST	0 = No, 1 = Yes	0: For bursts of 2 or more data beats, the AW/ARSIZE value is expected to be the maximum value supported by the DATA_WIDTH, which is the default interpretation when the AW/ARSIZE signal is absent on the AXI interface; for example, AWSIZE = 6 for DATA_WIDTH = 512 (64 bytes); otherwise such a transaction is considered to be an error. But AW/ARSIZE can be any smaller value when AW/ARLEN = 0, to indicate a single transfer with footprint smaller than DATA_WIDTH (narrow singles are okay). 1: AW/ARSIZE can be any protocol-compliant value, which can result in shortened address strides for multi-beat bursts. This might, in turn, incur additional burst-packing logic cost in interconnect IP (such as SmartConnect) and/or endpoint slave IP. Some endpoint master IP might be packaged with SUPPORTS_NARROW_BURST = 1 (default) if there is any risk or uncertainty whether the master produces narrow multi-beat bursts. Override this value only when certain that narrow bursts will never occur in a given system design, to avoid unnecessary burst processing logic.

Table 6: Advanced Settings Parameters (cont'd)

Vivado IDE Parameter Name	User Parameter Name	Range	Description
HAS BURST	HAS_BURST	0 = No, 1 = Yes	0: The endpoint master is expected to only produce INCR type bursts (AW/ARBURST = 1, or AW/ARBURST signal not present); otherwise such a transaction is considered to be an error. 1: The endpoint master might produce INCR or WRAP bursts (Xilinx does not support FIXED bursts). This might, in turn, incur additional logic cost in interconnect IP (such as SmartConnect) and/or endpoint slave IP. Some endpoint master IP can be packaged with HAS_BURST = 1 (default) if there is any risk or uncertainty whether the master produces WRAP bursts, such as when the master contains a processor that can issue a target-word-first burst. Override this value only when certain that WRAP bursts will never occur in a given system design, to avoid unnecessary burst processing logic.
HAS LOCK, HAS CACHE, HAS REGION, HAS QOS, HAS PROT, HAS WSTRB, HAS BRESP, HAS RRESP	HAS_LOCK, HAS_CACHE, HAS_REGION, HAS_QOS, HAS_PROT, HAS_WSTRB, HAS_BRESP, HAS_RRESP	0 = No, 1 = Yes	These parameters indicate whether the corresponding AXI signal pin is present on the endpoint master interface and/or whether the signal needs to be propagated to the slave. There is generally no logic costs associated with these signals other than the width of the payload.
MAX BURST LENGTH	MAX_BURST_LENGTH	1-256	Indicates the maximum AW/ARLEN value expected to be produced by the endpoint master. This metadata can be used by some IP to reduce burst processing logic. There is no error-checking performed by interconnect IP (such as SmartConnect) regarding this metadata value. The results of issuing a transaction with AW/ARLEN exceeding MAX_BURST_LENGTH is unpredictable.

Table 6: Advanced Settings Parameters (cont'd)

Vivado IDE Parameter Name	User Parameter Name	Range	Description
NUM READ THREADS, NUM WRITE THREADS	NUM_READ_THREADS, NUM_WRITE_THREADS	0-16	0: Applies only to a WRITE_ONLY or READ_ONLY endpoint master. 1: Referred to as "single-ordering", all read transactions or all write transactions, respectively, are handled as if they belong to the same thread, regardless of their ARID/AWID value, with no opportunity for out-of-order responses. In general, a sequence of transactions is only allowed to be issued to one endpoint slave at a time. Any transaction targeting a different slave gets stalled at the interconnect until all outstanding transactions complete, as part of the interconnect's "single-slave-per-ID" deadlock avoidance policy. (The AXI Register Slice IP does not stall AXI transfers for any reason.) >1: Referred to as "multi-threading", the set limit determines how many different ARID or AWID values, respectively, can be outstanding concurrently (even if ID_WIDTH provides for more values). In general, each thread is allowed to independently access one endpoint slave at a time, and there is opportunity for out-of-order responses among the different outstanding threads. Any transaction presenting a different ARID or AWID after the set limit is reached gets stalled at the interconnect until all outstanding transactions complete for any one thread, as part of the interconnect's thread-tracking logic. In general, increasing NUM_*THREADS increases the interconnect's thread-tracking logic cost. (The AXI Register Slice IP does not keep track of ID threads.)
NUM READ OUTSTANDING, NUM WRITE OUTSTANDING	NUM_READ_OUTSTANDING, NUM_WRITE_OUTSTANDING	0-256	Indicates the maximum number of read transactions or write transactions, respectively, expected to be produced by the endpoint master. (0 applies only to a WRITE_ONLY or READ_ONLY endpoint master.) This metadata can be used by some IP to reduce transaction processing logic. In general, reducing this metadata value does not actually prevent a master from issuing more transactions, nor does it prevent an interconnect IP from accepting or propagating more transactions. (NUM_*OUTSTANDING is not the same as ACCEPTANCE or ISSUING limits.)

AXI Interface Parameters

The following table lists the parameters that control the physical characteristics of the AXI interfaces of the core.

Table 7: AXI Interface Parameters

Parameter Name	Default Value	Format/Range	Description
ID_WIDTH ¹	0	Integer (0-32)	Width of all ID signals propagated by the core.
ADDR_WIDTH ¹	32	For AXI4 or AXI3: Integer (12-64); for AXI4-Lite: Integer (1-64)	Width of all addr signals.
DATA_WIDTH ¹	32	For AXI4 or AXI3: Integer (32, 64, 128, 256, 512, 1024); for AXI4-Lite: Integer (32, 64)	Data width of the Write and Read datapaths.
AWUSER_WIDTH ¹	0	Integer (0-1024)	Width of awuser signals (if any).
ARUSER_WIDTH ¹	0	Integer (0-1024)	Width of aruser signals (if any).
WUSER_WIDTH ¹	0	Integer (0-1024)	Width of wuser signals (if any).
RUSER_WIDTH ¹	0	Integer (0-1024)	Width of ruser signals (if any).
BUSER_WIDTH ¹	0	Integer (0-1024)	Width of buser signals (if any).
READ_WRITE_MODE ¹	READ_WRITE	String (READ_WRITE, READ_ONLY, WRITE_ONLY)	Enables read channels and/or write channels.
PROTOCOL ¹	AXI4	String (AXI4, AXI3, AXI4LITE)	Protocol of all interfaces.

Notes:

1. Automatically set by tools based on system connectivity.

Designing with the Core

This section includes guidelines and additional information to facilitate designing with the core.

Modes of Operation

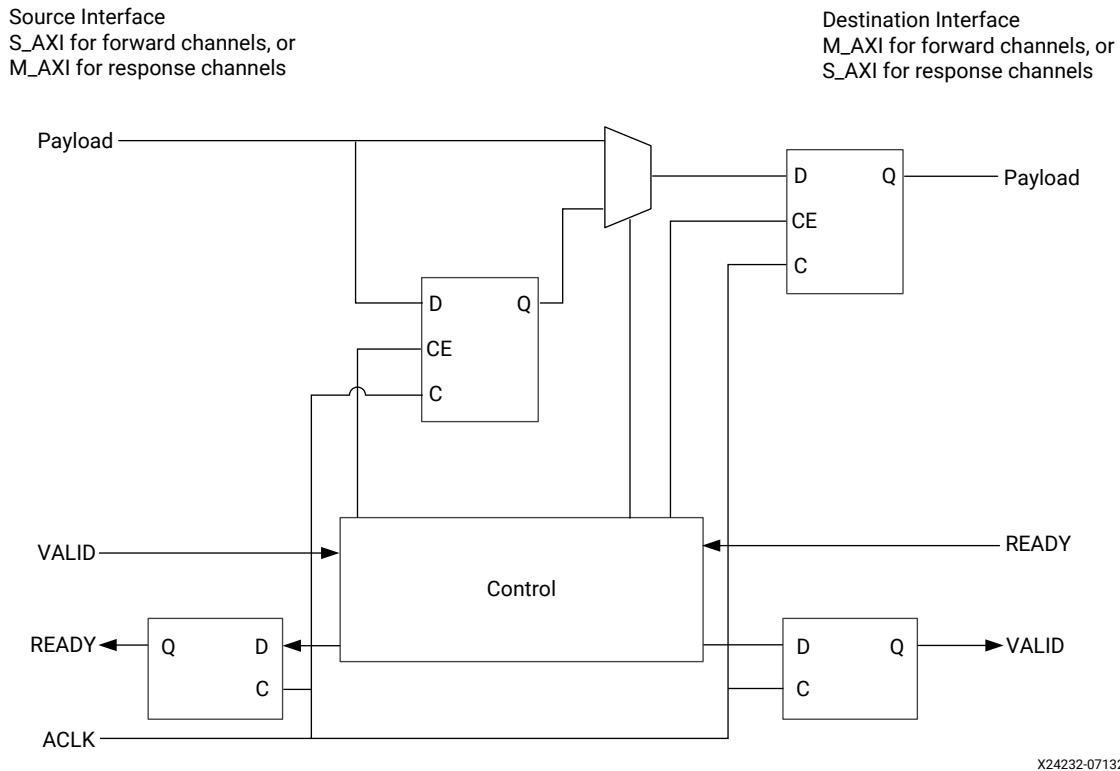
The following modes of the AXI Register Slice are available on each AXI channel.

Fully-Registered

Implemented as a two-deep FIFO buffer, this mode supports throttling by the channel source and/or channel destination as well as back-to-back transfers without incurring bubble cycles (up to 100% duty cycle). This mode is appropriate on W and R channels carrying bandwidth-critical AXI4 or AXI3 burst transfers.

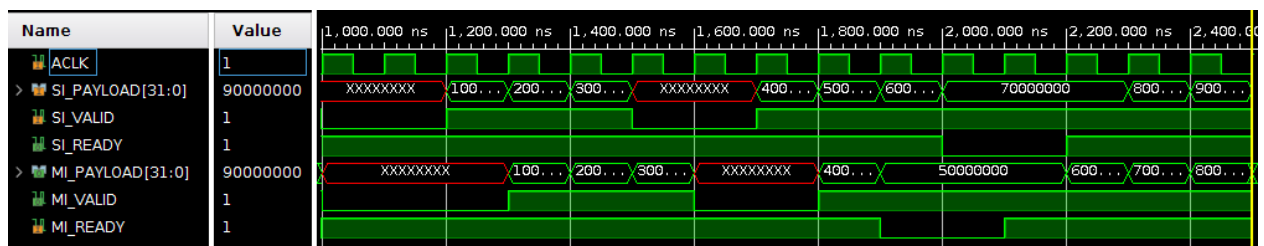
This mode is selected as "Full" in the configuration dialog. Fully-registered mode drives registered payload outputs and registered VALID and READY handshake outputs as shown in the figure.

Figure 2: Fully-Registered AXI Register Slice Diagram



The figure shows that the Fully-Registered mode introduces one latency cycle, but no bubble cycles.

Figure 3: Fully-Registered Mode Timing Diagram

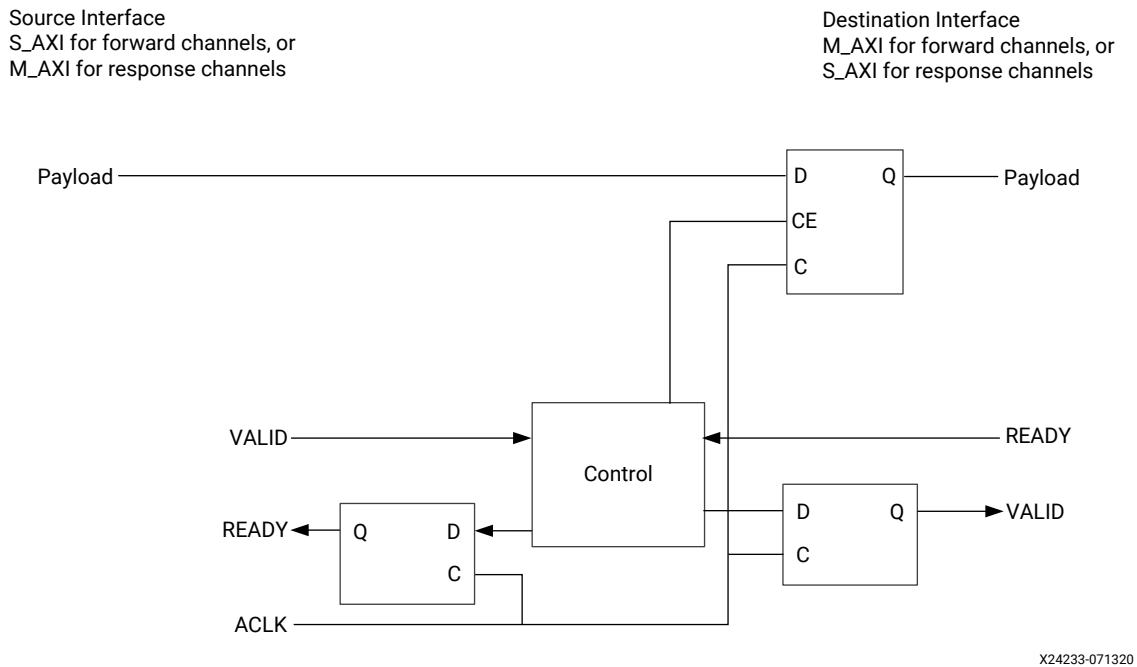


Light-Weight

Implemented as a simple one-stage pipeline register, this mode minimizes resources while isolating timing paths, but always incurs one bubble cycle following each transfer, resulting in a maximum duty cycle of 50%. This mode is appropriate on AW, AR and B channels, which normally do not require back-to-back transfers, and for all channels operating in AXI4-Lite protocol.

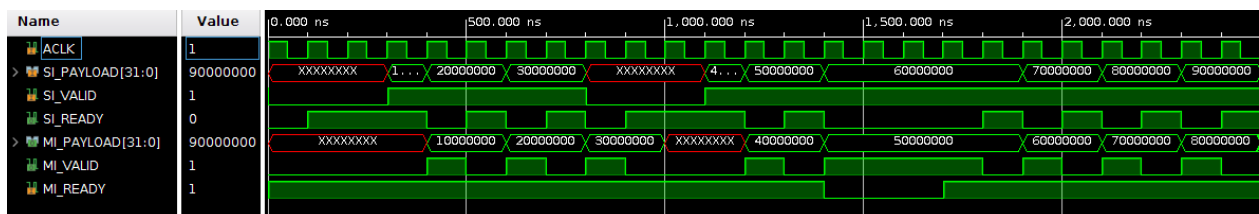
This mode is selected as "Light" in the configuration dialog. Light-weight mode drives registered payload outputs and registered VALID and READY handshake outputs as shown in the figure.

Figure 4: Light-Weight AXI Register Slice Diagram



The figure shows that the Light-Weight mode introduces one latency cycle, with one bubble cycle following each transfer.

Figure 5: Light-Weight Mode Timing Diagram

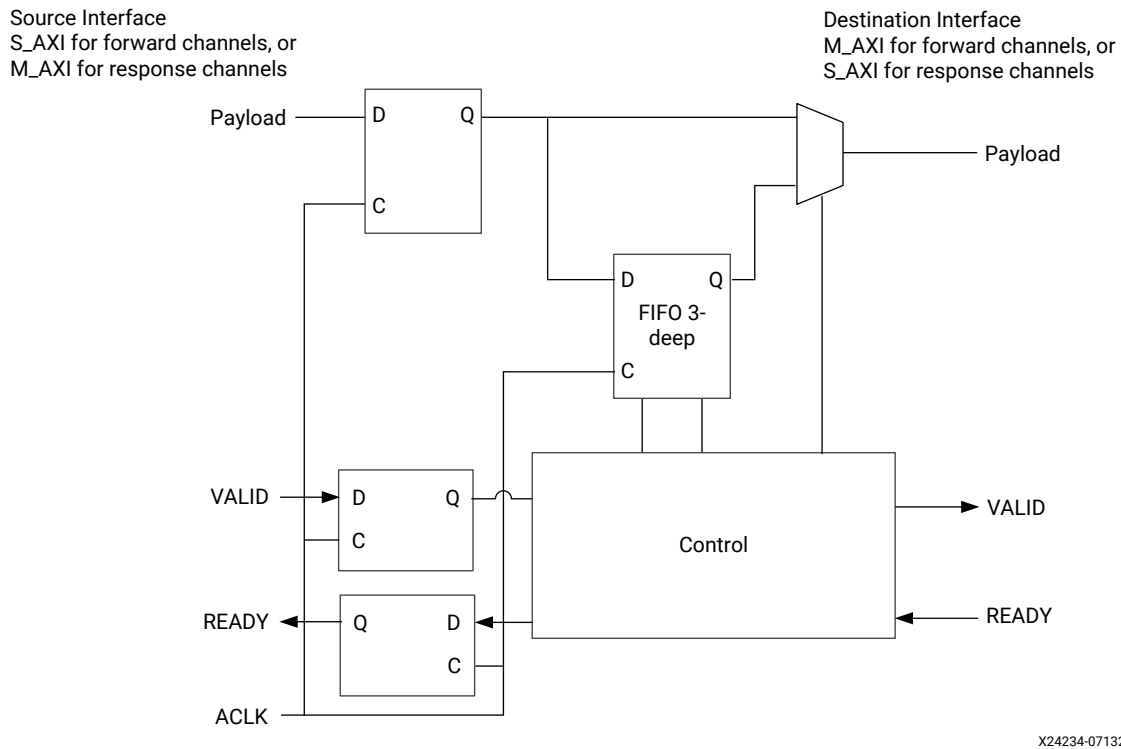


Registered Input

This mode is selected as SI Reg on forward-propagating channels and as MI Reg on response channels. Implemented as a 4-deep FIFO buffer, this mode passes the VALID handshake input and all payload inputs on the source side through simple flip-flops before applying any throttling logic. This mode is particularly useful to pipeline an AXI channel pathway originating from an adjacent SLR region in a SSI device. Similar to the Fully-Registered mode, this mode supports back-to-back transfers (100% duty cycle) with one cycle of forward latency. For more on parameters and options, see the related information below.

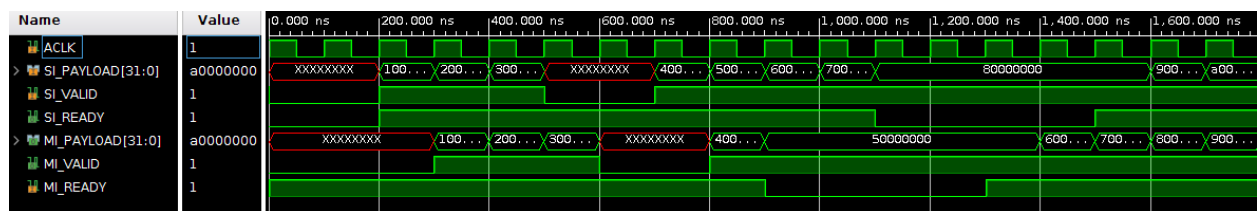
Registered Input mode samples payload inputs and the VALID handshake input into registers, and drives registered READY handshake output as shown in the figure.

Figure 6: Registered Input AXI Register Slice Diagram



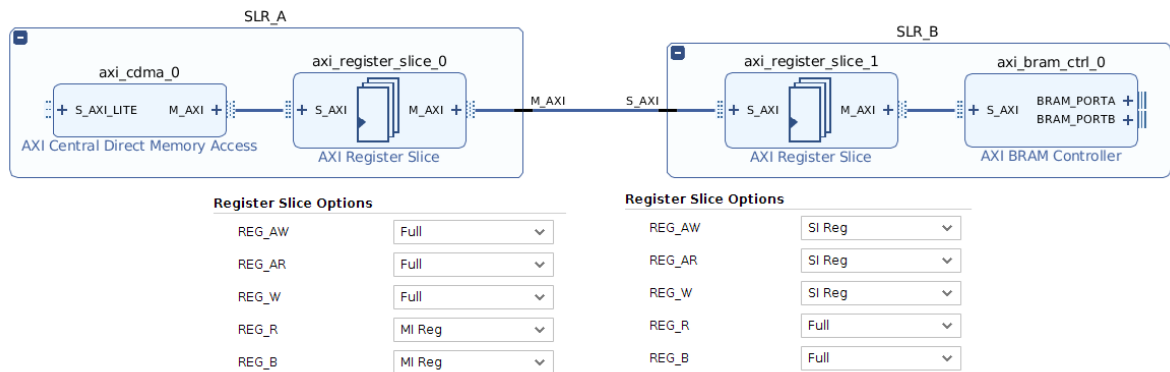
The figure shows that the registered input mode introduces one latency cycle (but no bubble cycles) in the source-to-destination payload and VALID transfer, and two cycles of latency (but no bubble cycles) in the destination-to-source READY handshake transfer.

Figure 7: Registered Input Mode Timing Diagram



When deploying two AXI Register Slice instances to cross a SLR boundary, configure each channel so that the source endpoint of the channel is fully-registered (or light-weight) and the destination endpoint has registered inputs (SI Reg or MI Reg). All signals crossing SLRs terminate at flip-flops, except the READY handshake input on each channel's source endpoint, which must directly drive handshake logic (see figure below).

Figure 8: Two AXI Register Slice Instances Diagram



Related Information

[AXI Register Slice Parameters](#)

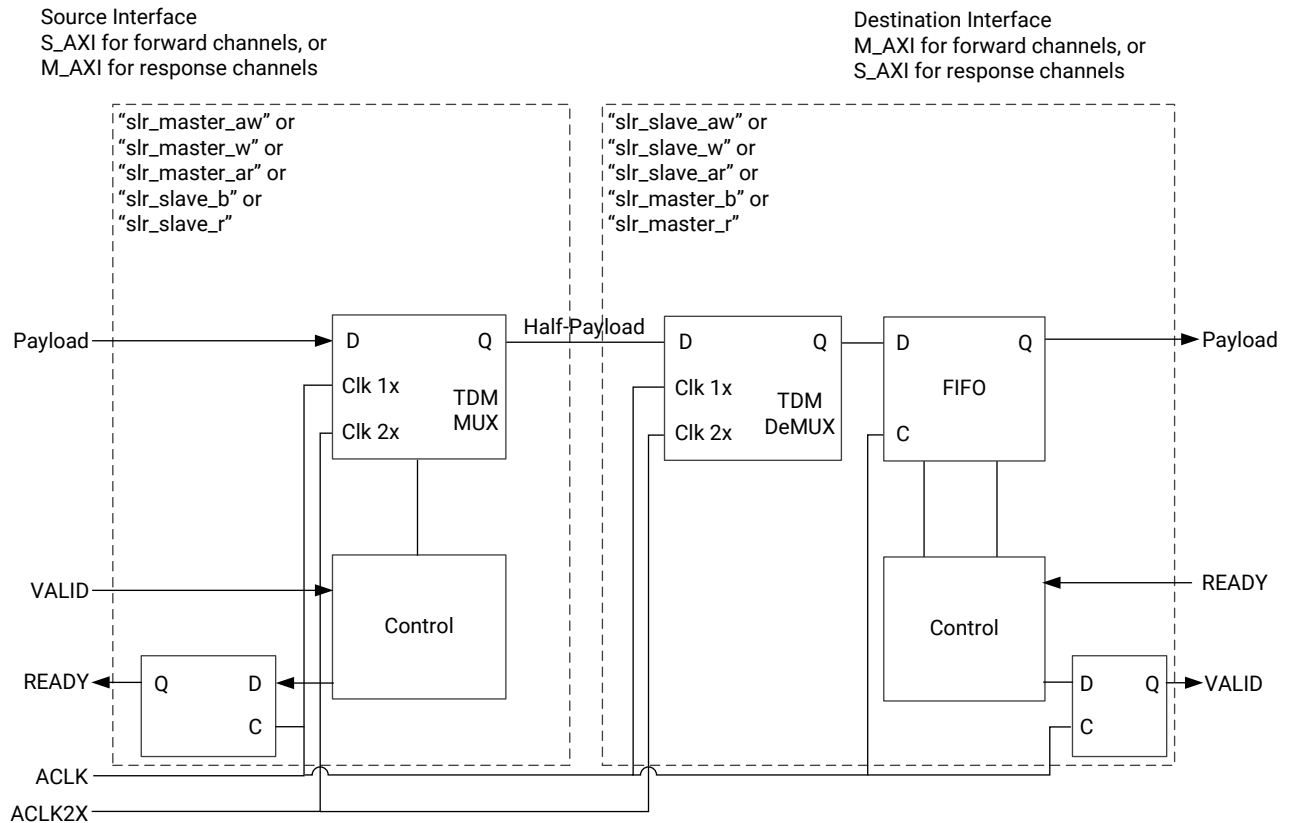
[Register Slice Options](#)

SLR TDM Crossing

This adds extra pipeline stages to optimally cross one SLR boundary in SSI devices. The SI interface of the AXI Register Slice and its connected AXI master device would then be located in one SLR, while the MI interface and its connected AXI slave device would be located in an adjacent SLR. The SLR-crossing pathway consumes half the number of payload wires and propagates the cross-SLR signals at twice the frequency of the AXI interfaces. Configuring any AXI channel in this mode requires driving the `ac1k2x` clock input with a double-frequency edge-aligned clock signal.

SLR TDM Crossing mode drives registered payload outputs and registered VALID and READY handshake outputs as shown in the figure. The submodule instance names shown can be used in cell-name patterns when writing floorplanning (Pblock) constraints, as needed, to enforce proper SLR placement.

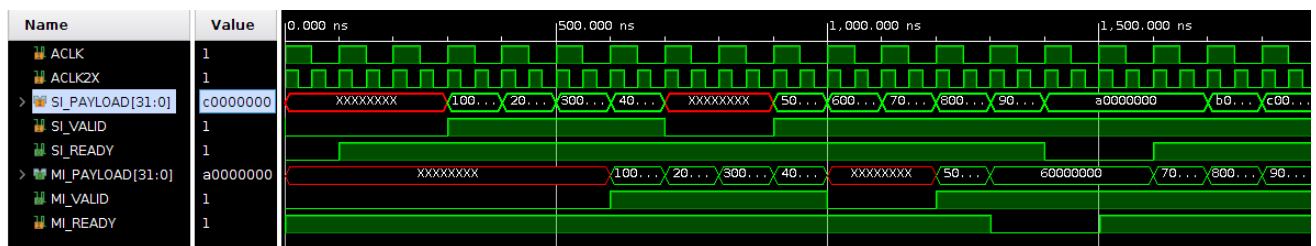
Figure 9: SLR TDM Crossing AXI Register Slice Diagram



X24235-071420

The figures shows that the SLR TDM Crossing mode introduces three ACLK latency cycles (but no bubble cycles) in the source-to-destination payload and VALID transfer, and one ACLK latency cycle (but no bubble cycles) in the destination-to-source READY handshake transfer.

Figure 10: SLR TDM Crossing Mode Timing Diagram

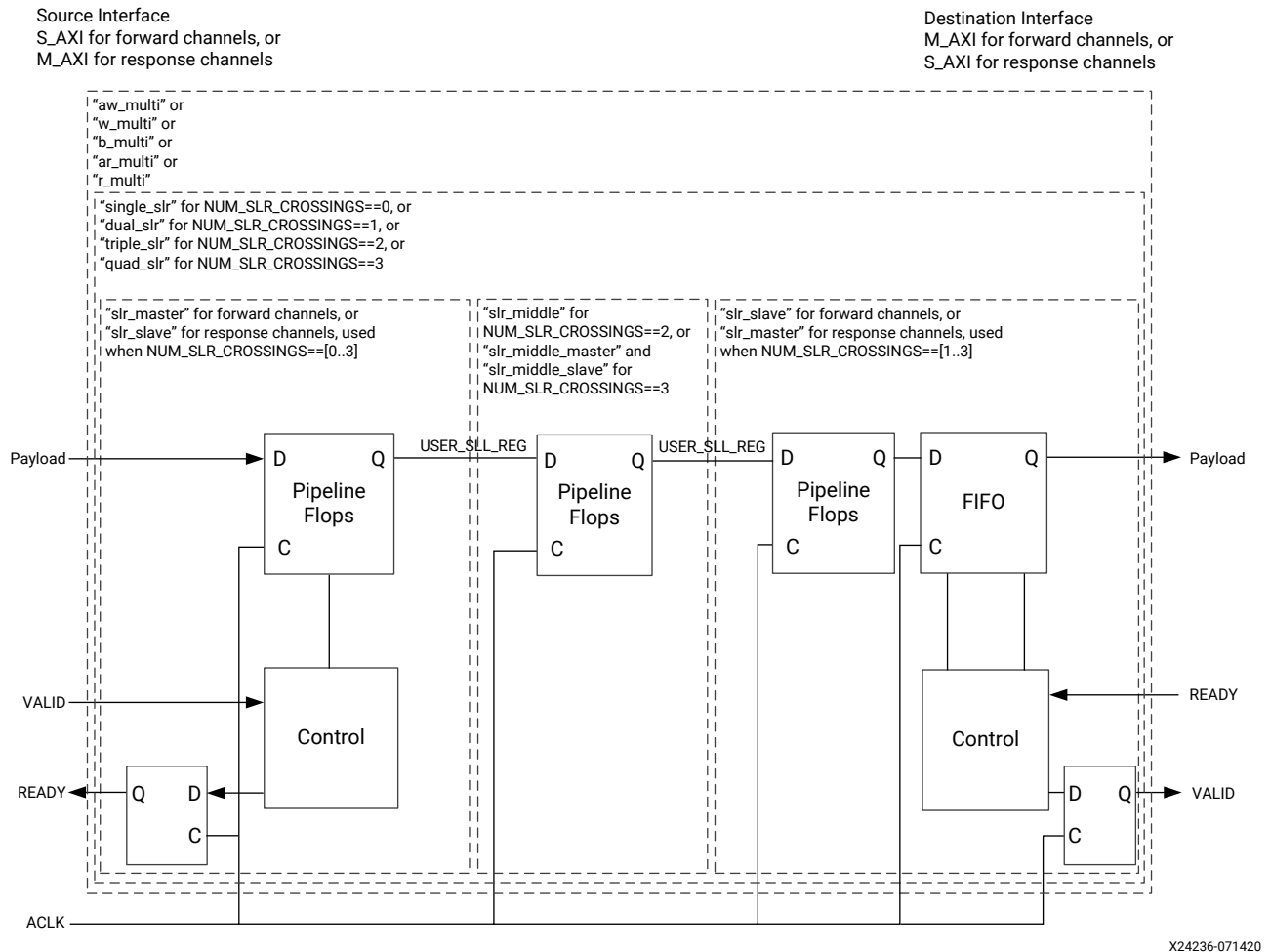


Multi SLR Crossing

This supports spanning zero or more SLR boundaries using a single AXI Register Slice instance. Also, provides a selectable number of intermediate pipeline stages within each SLR to help close timing. All SLR crossings within the core are flop-to-flop with fanout = 1. See Constraining the Core section in the Related Information for floorplanning guidance.

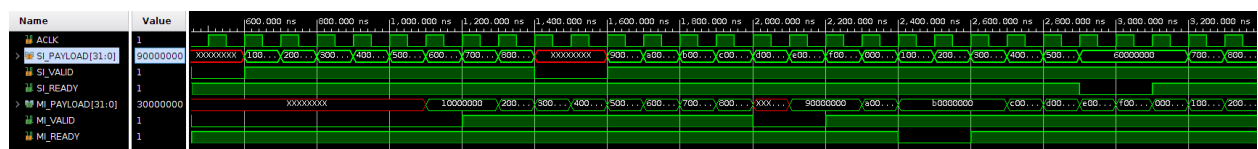
Multi SLR Crossing mode drives registered payload outputs and registered VALID and READY handshake outputs as shown in the figure. The submodule instance names shown can be used in cell-name patterns when writing floorplanning (Pblock) constraints, as needed, to enforce proper SLR placement.

Figure 11: Multi SLR Crossing AXI Register Slice Diagram



The figure shows that the Multi SLR Crossing mode introduces a variable number of latency cycles, as determined by the various SLR Crossing parameters, but no bubble cycles.

Figure 12: Multi SLR Crossing Mode Timing Diagram



Related Information

[Constraining the Core](#)

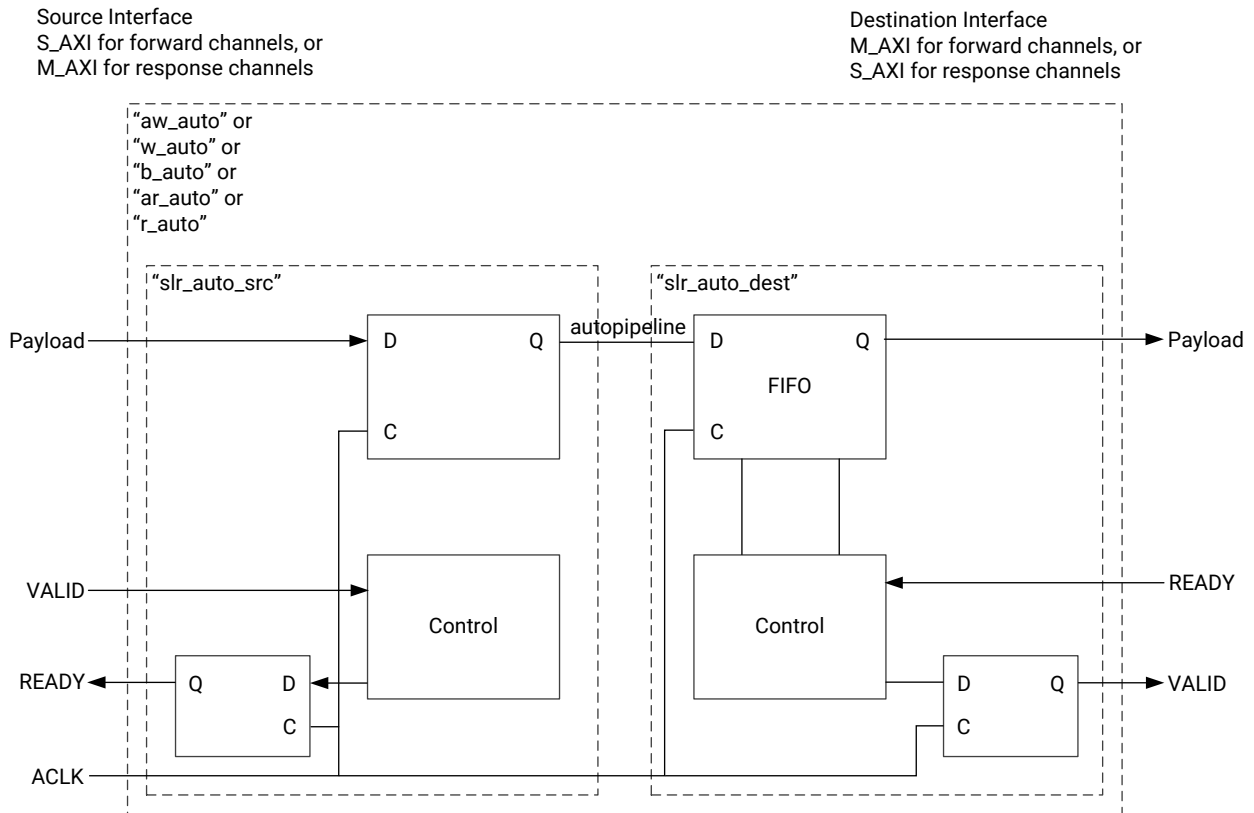
Auto-Pipeline Insertion

This mode is enabled by selecting **Multi SLR Crossing** for one or more AXI channels and checking the box **Use timing-driven pipeline insertion** in the configuration dialog. Like Multi SLR Crossing mode, this mode supports spanning zero or more SLR boundaries using a single AXI Register Slice instance. However, the number of pipeline stages inserted for both SLR-crossing and intra-SLR pipelining is determined automatically during the physical optimization stage of design implementation, as needed to satisfy pathway timing. No bubble cycles are incurred.

Note: Pipeline register resources and resulting latency might vary after each implementation run, and resulting latency will not be exhibited during behavioral simulation.

Auto-Pipeline Insertion mode drives registered payload outputs and registered VALID and READY handshake outputs as shown in the figure. The submodule instance names shown can be used in cell-name patterns when writing floorplanning (Pblock) constraints, as needed, to enforce proper SLR placement of source and destination submodules. (No intermediate SLRs need to be floorplanned.)

Figure 13: Auto-Pipeline Insertion AXI Register Slice Diagram



X24231-071520

Bypass

Directly connects the SI to the MI.

Legacy Modes

Additionally, the following legacy modes of the AXI Register Slice are also available on each AXI channel.

Forward

Only the payload and VALID handshake outputs are registered.

Reverse

Only the READY handshake output is registered.

Inputs

Only payload and handshake inputs are registered.

SLR Crossing

This mode adds extra pipeline stages to optimally cross an SLR boundary in SSI devices. The SI interface of the AXI Register Slice and its connected AXI master device would then be located in one SLR, while the MI interface and its connected AXI slave device would be located in an adjacent SLR. All SLR crossings within the core are flop-to-flop with fanout = 1. This mode has been superseded by the Multi-SLR Crossing mode. See Constraining the Core section for floorplanning guidance.

Related Information

[Constraining the Core](#)

General Design Guidelines

Use the Example Design

The AXI Register Slice core created by the Vivado® design tool is delivered with an example design that can be simulated or implemented in a device. This design can be used as a starting point for your own design or can be used to sanity-check your application in the event of difficulty. See the Example Design content for information about using and customizing the example designs for the core.

Related Information

[Example Design](#)

Make Only Allowed Modifications

You should not modify the core. Any modifications can have adverse effects on system timing and protocol compliance. Supported user configurations of the core can only be made by selecting the options in the customization IP dialog box when the core is generated.

Clocking

The S_AXI and M_AXI interfaces are both synchronized by the `aclk` input. The AXI Register Slice IP does not perform clock conversion.

When SLR TDM Crossing mode is selected for any AXI channel, the core also enables its `ACLK2X` input and requires that it be driven by a clock produced by the same source as `ACLK` with exactly 2x the `ACLK` frequency and zero-phase (edge-aligned).

Resets

The `aresetn` input can be tied High (inactive) if no "soft" reset is required. AXI handshake outputs remain deasserted until the device recovers from power-on reset, at which time handshake inputs are sampled. Behavioral simulation initializes to the same state as the device following power-on.

The `aresetn` input, if used, must be synchronized by `aclk`.

This IP deasserts all valid and ready outputs shortly after `aresetn` is sampled active, and for the duration of the `aresetn` pulse. AXI protocol requires that the connected master also deasserts all valid outputs during reset (until after `aresetn` is sampled inactive). Slaves must not assert response-channel valid outputs until after they receive a command from a master. It is also strongly recommended that slave IP deassert their ready outputs until after reset. This avoids inadvertently signaling a transfer completion in case a connected IP recovers from reset during an earlier cycle and asserts are valid.

There is no requirement that the assertion or deassertion of `aresetn` be observed during the same cycle or in any relative order among this IP and its connected master and slave. It is, however, required that the cycles during which reset is applied to this IP and its connected master and slave overlap. This IP does not support independent reset domains. If the master or slave device connected to this IP is reset, then all connected devices must be reset concurrently.



RECOMMENDED: As a general design guideline, Xilinx recommends asserting system `aresetn` signals for a minimum of 16 clock cycles, as that is known to satisfy the reset requirements.

Design Flow Steps

This section describes customizing and generating the core, constraining the core, and the simulation, synthesis, and implementation steps that are specific to this IP core. More detailed information about the standard Vivado® design flows and the IP integrator can be found in the following Vivado Design Suite user guides:

- *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#))
- *Vivado Design Suite User Guide: Designing with IP* ([UG896](#))
- *Vivado Design Suite User Guide: Getting Started* ([UG910](#))
- *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#))

Customizing and Generating the Core

This section includes information about using Xilinx® tools to customize and generate the core in the Vivado® Design Suite.

If you are customizing and generating the core in the Vivado IP integrator, see the *Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator* ([UG994](#)) for detailed information. IP integrator might auto-compute certain configuration values when validating or generating the design. To check whether the values do change, see the description of the parameter in this chapter. To view the parameter value, run the `validate_bd_design` command in the Tcl console.

You can customize the IP for use in your design by specifying values for the various parameters associated with the IP core using the following steps:

1. Select the IP from the IP catalog.
2. Double-click the selected IP or select the Customize IP command from the toolbar or right-click menu.

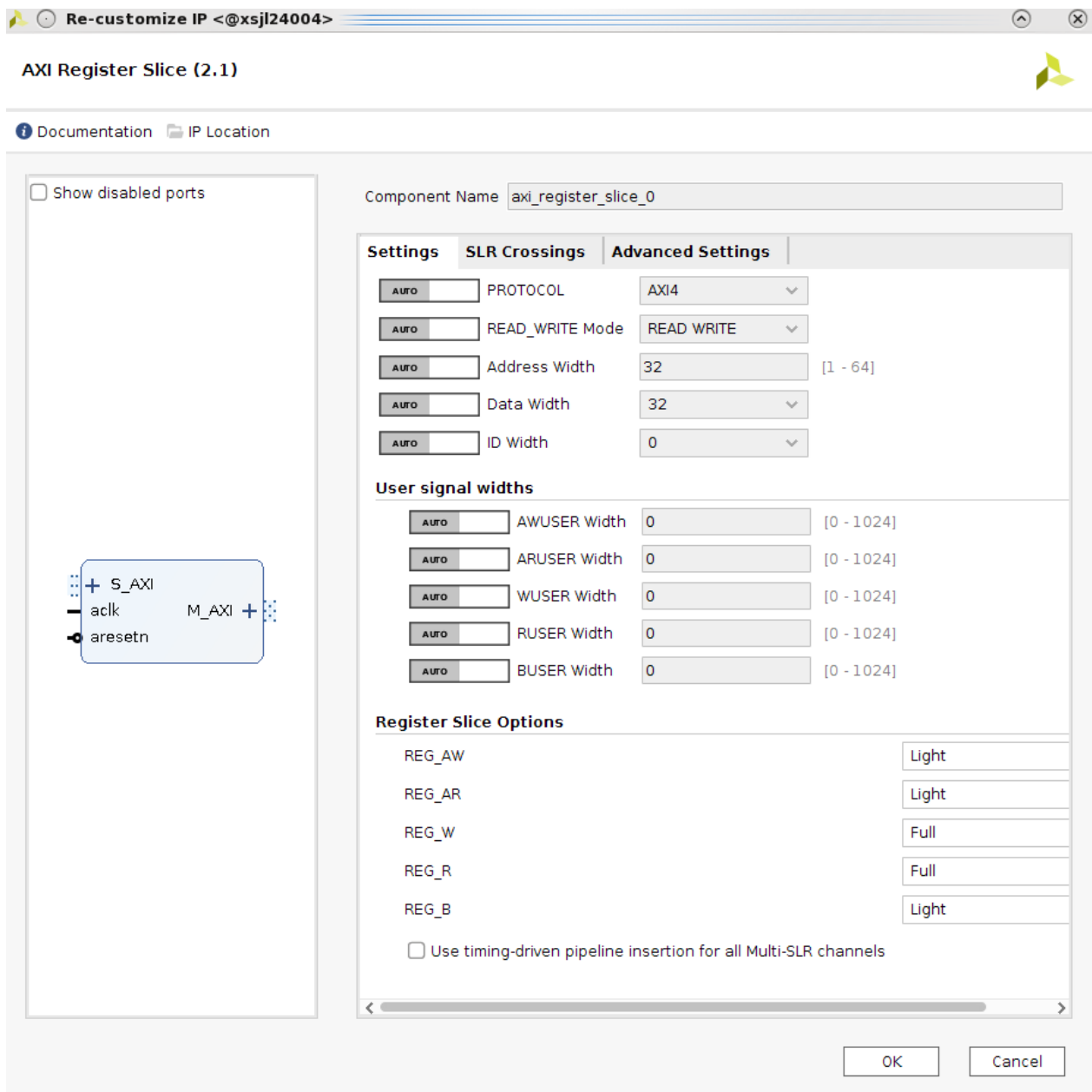
For parameters showing an Auto/Manual selector widget to the left, values are normally filled in automatically according to the characteristics of connected endpoint IP when you validate your BD design. By changing the widget to Manual, you can override the parameter value which, in turn, can override corresponding characteristics of the connected endpoint IP or interconnect IP, after you validate your BD design.

For details, see the *Vivado Design Suite User Guide: Designing with IP* (UG896) and the *Vivado Design Suite User Guide: Getting Started* (UG910).

Figures in this chapter are illustrations of the Vivado IDE. The layout depicted here might vary from the current version.

AXI Register Slice Settings Tab

Figure 14: AXI Register Slice Settings Tab



PROTOCOL

- **Description:** Protocol of all interfaces. Automatically set by tools based on system connectivity.
- **User Parameter Name:** PROTOCOL
- **Format/Range:** String (AXI4, AXI3, AXI4LITE)
- **Default Value:** AXI4

READ_WRITE Mode

- **Description:** Enables read channels and/or write channels. Automatically set by tools based on system connectivity.
- **User Parameter Name:** READ_WRITE_MODE
- **Format/Range:** String (READ_WRITE, READ_ONLY, WRITE_ONLY)
- **Default Value:** READ_WRITE

Address Width

- **Description:** Width of all `addr` signals. Automatically set by tools based on system connectivity.
- **User Parameter Name:** ADDR_WIDTH
- **Format/Range:** For AXI4 or AXI3: Integer (12-64); for AXI4-Lite: Integer (1-64)
- **Default Value:** 32

Data Width

- **Description:** Data width of the Write and Read datapaths. Automatically set by tools based on system connectivity.
- **User Parameter Name:** DATA_WIDTH
- **Format/Range:** For AXI4 or AXI3: Integer (32, 64, 128, 256, 512, 1024); for AXI4-Lite: Integer (32, 64)
- **Default Value:** 32

ID Width

- **Description:** Width of all ID signals propagated by the core. Automatically set by tools based on system connectivity.

- **User Parameter Name:** ID_WIDTH
- **Format/Range:** Integer (0-32)
- **Default Value:** 0

User Signal Widths

- AWUSER/ARUSER/WUSER/RUSER/BUSER WIDTH
 - **Description:** Width of corresponding user signals (if any). Automatically set by tools based on system connectivity.
 - **User Parameter Name:** AWUSER_WIDTH, ARUSER_WIDTH, WUSER_WIDTH, RUSER_WIDTH, BUSER_WIDTH
 - **Format/Range:** Integer (0-1024)
 - **Default Value:** 0

Register Slice Options

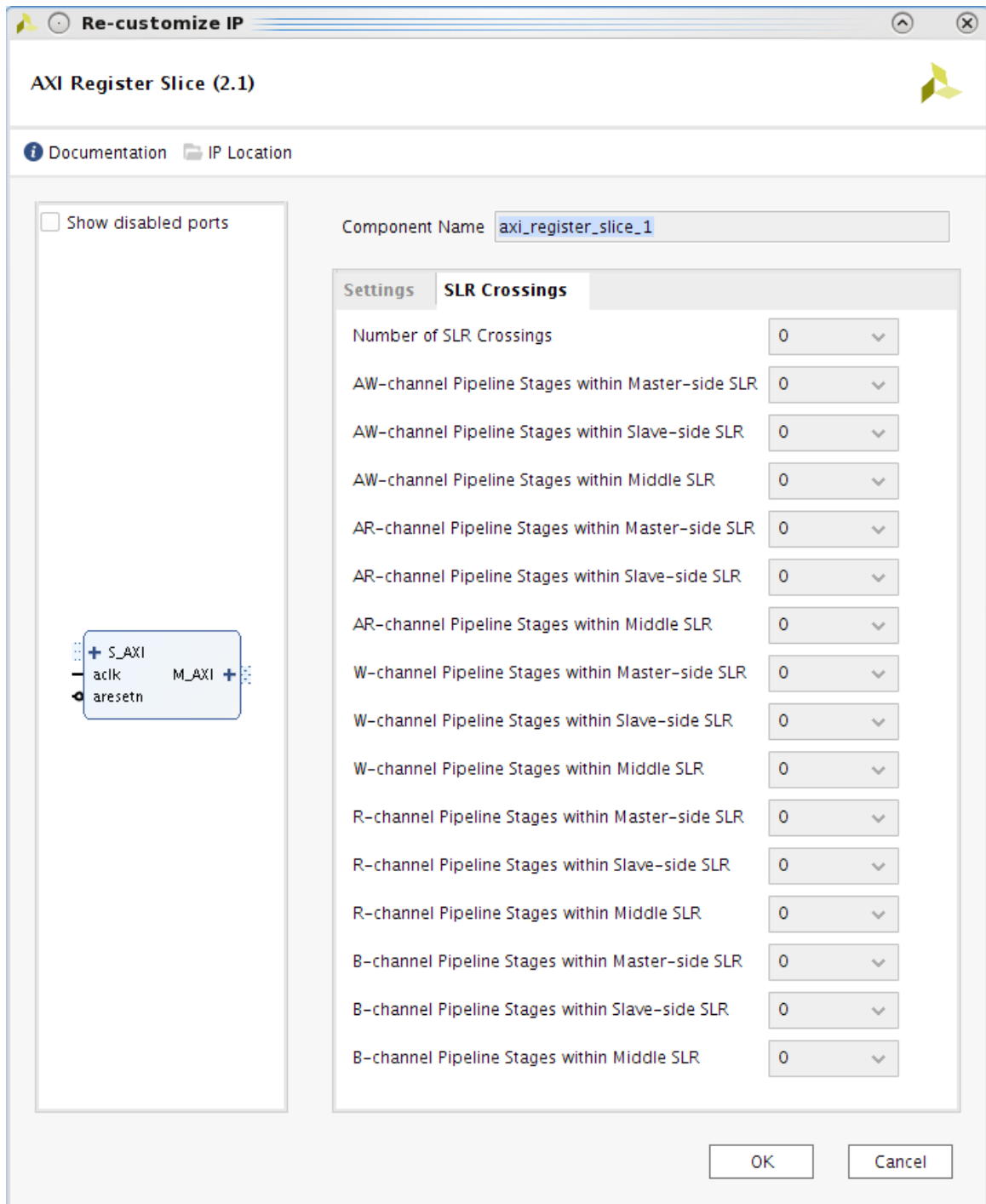
- REG_AW
 - **Description:** Mode of channel register slice.
 - **User Parameter Name:** REG_AW
 - **Format/Range:** Integer (See [Table 9: AXI Register Slice Options Parameter Values](#))
 - **Default Value:** Light
- REG_AR
 - **Description:** Mode of channel register slice.
 - **User Parameter Name:** REG_AR
 - **Format/Range:** Integer (See [Table 9: AXI Register Slice Options Parameter Values](#))
 - **Default Value:** Light
- REG_W
 - **Description:** Mode of channel register slice.
 - **User Parameter Name:** REG_W
 - **Format/Range:** Integer (See [Table 9: AXI Register Slice Options Parameter Values](#))
 - **Default Value:** Light if PROTOCOL = AXI4LITE, otherwise Full
- REG_R
 - **Description:** Mode of channel register slice.
 - **User Parameter Name:** REG_R

- **Format/Range:** Integer (See [Table 9: AXI Register Slice Options Parameter Values](#))
- **Default Value:** Light if PROTOCOL = AXI4LITE, otherwise Full
- REG_B
 - **Description:** Mode of channel register slice.
 - **User Parameter Name:** REG_B
 - **Format/Range:** Integer (See [Table 9: AXI Register Slice Options Parameter Values](#))
 - **Default Value:** Light

SLR Crossings Tab

The properties in this tab are enabled only when Multi SLR Crossing without Auto-Pipeline Insertion is selected for any AXI channel.

Figure 15: SLR Crossings Tab



- Number of SLR Crossings
 - **Description:** Select the number of SLR boundaries to be crossed within the core.
 - **User Parameter Name:** NUM_SLR_CROSSINGS

- **Format/Range:** Integer (0-3)
- **Default Value:** 0
- AW/AR/W/R/B-channel Pipeline Stages within Master-side SLR
 - **Description:** Select the number of additional pipeline stages to insert within the master-side SLR (between the SLR boundary and the SI interface).
 - **User Parameter Name:** PIPELINES_MASTER_AW, PIPELINES_MASTER_AR, PIPELINES_MASTER_W, PIPELINES_MASTER_R, PIPELINES_MASTER_B
 - **Format/Range:** Integer (0-4)
 - **Default Value:** 0
- AW/AR/W/R/B-channel Pipeline Stages within Slave-side SLR
 - **Description:** Select the number of additional pipeline stages to insert within the slave-side SLR (between the SLR boundary and the MI interface).
 - **User Parameter Name:** PIPELINES_SLAVE_AW, PIPELINES_SLAVE_AR, PIPELINES_SLAVE_W, PIPELINES_SLAVE_R, PIPELINES_SLAVE_B
 - **Format/Range:** Integer (0-4)
 - **Default Value:** 0
- AW/AR/W/R/B-channel Pipeline Stages within Middle SLR
 - **Description:** Select the number of additional pipeline stages to insert within the middle SLR (between the two SLR boundaries). Enabled only when Number of SLR Crossings is > 1.
 - **User Parameter Name:** PIPELINES_MIDDLE_AW, PIPELINES_MIDDLE_AR, PIPELINES_MIDDLE_W, PIPELINES_MIDDLE_R, PIPELINES_MIDDLE_B
 - **Format/Range:** Integer (0-4)
 - **Default Value:** 0

Advanced Settings Tab

Figure 16: Advanced Settings Tab

Re-customize IP <@xsjl24004>

AXI Register Slice (2.1)

Documentation IP Location

Component Name

Settings SLR Crossings **Advanced Settings**

optional choice

AUTO	WUSER BITS PER BYTE	0	[0 - 32]
AUTO	RUSER BITS PER BYTE	0	[0 - 32]
AUTO	NUM READ THREADS	0	[0 - 16]
AUTO	NUM WRITE THREADS	0	[0 - 16]
AUTO	NUM READ OUTSTANDING	1	[0 - 256]
AUTO	NUM WRITE OUTSTANDING	1	[0 - 256]
AUTO	MAX BURST LENGTH	1	[1 - 256]
AUTO	SUPPORTS NARROW BURST	1	▼
AUTO	HAS BURST	1	▼
AUTO	HAS LOCK	1	▼
AUTO	HAS CACHE	1	▼
AUTO	HAS REGION	1	▼
AUTO	HAS PROT	1	▼
AUTO	HAS QOS	1	▼
AUTO	HAS WSTRB	1	▼
AUTO	HAS BRESP	1	▼
AUTO	HAS RRESP	1	▼
Reserved Mode		Register Slice ▼	

OK Cancel

- WUSER BITS PER BYTE, RUSER BITS PER BYTE
 - **Description:** Specifies the number of user bits in the WUSER or RUSER port that are associated with each data byte on the WDATA or RDATA port.
 - **User Parameter Name:** WUSER_BITS_PER_BYTE, RUSER_BITS_PER_BYTE
 - **Format/Range:** Integer (0-128)

- **Default Value:** 0
- NUM READ THREADS, NUM WRITE THREADS
 - **Description:** Specifies the number of concurrent threads of AXI transactions that are supported on the given interface.
 - **User Parameter Name:** NUM_READ_THREADS, NUM_WRITE_THREADS
 - **Format/Range:** Integer (1-16)
 - **Default Value:** 1
- NUM READ OUTSTANDING, NUM WRITE OUTSTANDING
 - **Description:** Specifies the number of read or write AXI transactions that can be in progress (pipelined) at the given AXI interface.
 - **User Parameter Name:** NUM_READ_OUTSTANDING, NUM_WRITE_OUTSTANDING
 - **Format/Range:** Integer (0-256)
 - **Default Value:** 1
- MAX BURST LENGTH
 - **Description:** Specifies the longest burst length supported at the given AXI interface.
 - **User Parameter Name:** MAX_BURST_LENGTH
 - **Format/Range:** Integer (1-256 if PROTOCOL==AXI4, 1-16 if PROTOCOL==AXI3, 1 if PROTOCOL==AXI4LITE)
 - **Default Value:** 1
- SUPPORTS NARROW BURST
 - **Description:** Specifies whether the interface supports burst transactions with a size smaller than the native data width of the interface.
 - **User Parameter Name:** SUPPORTS_NARROW_BURST
 - **Format/Range:** Integer (0-1)
 - **Default Value:** 1
- HAS BRESP
 - **Description:** Specifies whether the interface contains the BRESP port.
 - **User Parameter Name:** HAS_BRESP
 - **Format/Range:** Integer (0-1)
 - **Default Value:** 1
- HAS BURST
 - **Description:** Specifies whether the interface contains the A*BURST ports. If the interface omits the A*BURST ports, it indicates that only INCR type burst transactions are supported.

- **User Parameter Name:** HAS_BURST
- **Format/Range:** Integer (0-1)
- **Default Value:** 1
- HAS CACHE
 - **Description:** Specifies whether the port contains the A*CACHE ports.
 - **User Parameter Name:** HAS_CACHE
 - **Format/Range:** Integer (0-1)
 - **Default Value:** 1
- HAS LOCK
 - **Description:** Specifies whether the interface contains the A*LOCK ports.
 - **User Parameter Name:** HAS_LOCK
 - **Format/Range:** Integer (0-1)
 - **Default Value:** 1
- HAS PROT
 - **Description:** Specifies whether the interface contains the A*PROT ports.
 - **User Parameter Name:** HAS_PROT
 - **Format/Range:** Integer (0-1)
 - **Default Value:** 1
- HAS QOS
 - **Description:** Specifies whether the interface contains the A*QOS ports.
 - **User Parameter Name:** HAS_QOS
 - **Format/Range:** Integer (0-1)
 - **Default Value:** 1
- HAS REGION
 - **Description:** Specifies whether the interface contains the A*REGION ports.
 - **User Parameter Name:** HAS_REGION
 - **Format/Range:** Integer (0-1)
 - **Default Value:** 1
- HAS RRESP
 - **Description:** Specifies whether the interface contains the RRESP port.
 - **User Parameter Name:** HAS_RRESP

- **Format/Range:** Integer (0-1)
- **Default Value:** 1
- HASWSTRB
 - **Description:** Specifies whether the interface contains theWSTRB port.
 - **User Parameter Name:** HAS_WSTRB
 - **Format/Range:** Integer (0-1)
 - **Default Value:** 1

User Parameters

The following table shows the relationship between the fields in the Vivado® IDE and the user parameters (which can be viewed in the Tcl Console).

Constraining the AXI Register Slice submodules might be necessary if your implementation resulted in SLR crossings anywhere other than the intended pipelined pathway within the AXI Register Slice, typically resulting in timing-critical paths.

Table 8: User Parameters

Vivado IDE Parameter/ Value ¹	User Parameter/Value	Default Value
REG_AW	REG_AW	Light
REG_W	REG_W	Full if PROTOCOL = AXI4 or AXI3, Light if PROTOCOL = AXI4LIGHT
REG_B	REG_B	Light
REG_AR	REG_AR	Light
REG_R	REG_R	Full if PROTOCOL = AXI4 or AXI3, Light if PROTOCOL = AXI4LIGHT
Use timing-driven pipeline insertion	USE_AUTOPIPELINING	0

Notes:

1. Parameter values are listed in the table where the Vivado IDE parameter value differs from the user parameter value. Such values are shown in this table as indented below the associated parameter.

The following table shows the parameter values for the AXI Register Slice options.

Table 9: AXI Register Slice Options Parameter Values

Vivado IDE Parameter Value	User Parameter Value	Description
Bypass	0	Directly connects the SI to the MI
Forward	2	Only the payload and VALID handshake outputs are registered (legacy mode)
Full	1	Supports back-to-back transfers without incurring bubble cycles

Table 9: AXI Register Slice Options Parameter Values (cont'd)

Vivado IDE Parameter Value	User Parameter Value	Description
Inputs	6	Only payload and handshake inputs are registered (legacy mode)
Light	7	Simple one-stage pipeline register, incurs one bubble cycle following each transfer
Multi SLR Crossing	15	Supports spanning zero or more SLR boundaries using a single instance, plus a variable number of intermediate pipeline stages per SLR
Reverse	3	Only the READY handshake output is registered (legacy mode)
SI_Reg (for REG_AW, REG_W and REG_AR), or MI_Reg (for REG_B and REG_R)	9	Passes VALID and payload inputs on the source side through simple flip-flops, used to pipeline an AXI channel pathway originating from an adjacent SLR
SLR Crossing	10	Adds pipeline stages to optimally cross a single SLR boundary (legacy mode)
SLR TDM Crossing	11	Adds pipeline stages to optimally cross a single SLR boundary, consuming half the number of payload wires; requires 2x clock

Output Generation

For details, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)).

Constraining the Core

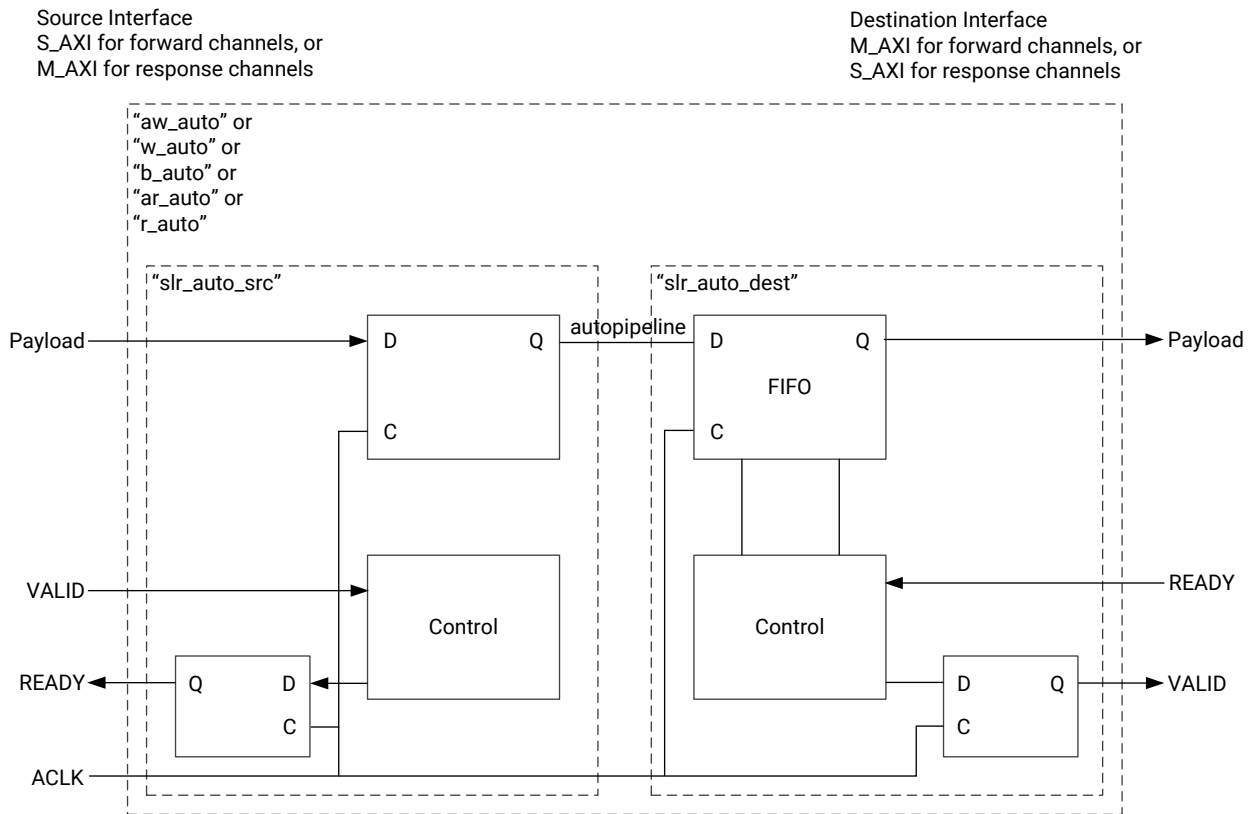
Required Constraints

When using the AXI Register Slice core in either the SLR Crossing, SLR TDM Crossing or Multi SLR Crossing mode, constraints must be applied to explicitly floorplan the SI-side and MI-side submodules of the core into the same SLRs as the connected master and slave endpoints. This will ensure that the SLR crossing(s) will take place between the intended flop-to-flop, unit-fanout, internal wires across all payload and handshake pathways within the core.

In most configurations, after synthesis, all logic and registers that should be placed into the master-side SLR (where the AXI master connected to the SI interface is located) will contain the cell name pattern `*slr_master*`. All logic and registers that should be placed into the slave-side SLR (where the AXI slave connected to the MI interface is located) will contain the cell name pattern `*slr_slave*`. When spanning three SLRs, all logic and registers that should be placed into the middle SLR will contain the cell name pattern `*slr_middle*`. Constraints that combine the instance name of the Register Slice and any of these submodule name patterns can then be used to group all cells in the core into their respective Pblocks for floorplanning.

For Auto-Pipeline Insertion mode, see the following figure for applicable submodule name patterns.

Figure 17: Auto-Pipeline Insertion Diagram



X24231-071520

In the following example, an AXI Register Slice instance named `my_reg` is configured in Multi SLR Crossing mode (all channels) to cross two SLR boundaries that exist in the target device. One of the boundaries exists between row Y4 (the top of the lower SLR) and row Y5 (the bottom of the middle SLR). The other boundary exists between row Y9 (the top of the middle SLR) and row Y10 (the bottom of the upper SLR).

```
create_pblock lower_slr
add_cells_to_pblock [get_pblocks lower_slr] [get_cells -hierarchical -
filter "NAME=~*my_reg*slr_master*"]
resize_pblock [get_pblocks lower_slr] -add SLR0
create_pblock center_slr
add_cells_to_pblock [get_pblocks center_slr] [get_cells -hierarchical -
filter "NAME=~*my_reg*slr_middle*"]
resize_pblock [get_pblocks center_slr] -add SLR1
create_pblock upper_slr
add_cells_to_pblock [get_pblocks upper_slr] [get_cells -hierarchical -
filter "NAME=~*my_reg*slr_slave*"]
resize_pblock [get_pblocks upper_slr] -add SLR2
```

Device, Package, and Speed Grade Selections

This section is not applicable for this IP core.

Clock Frequencies

This section is not applicable for this IP core.

Clock Management

This section is not applicable for this IP core.

Clock Placement

This section is not applicable for this IP core.

Banking

This section is not applicable for this IP core.

Transceiver Placement

This section is not applicable for this IP core.

I/O Standard and Placement

This section is not applicable for this IP core.

Simulation

For comprehensive information about Vivado® simulation components, as well as information about using supported third-party tools, see the *Vivado Design Suite User Guide: Logic Simulation* ([UG900](#)).

Synthesis and Implementation

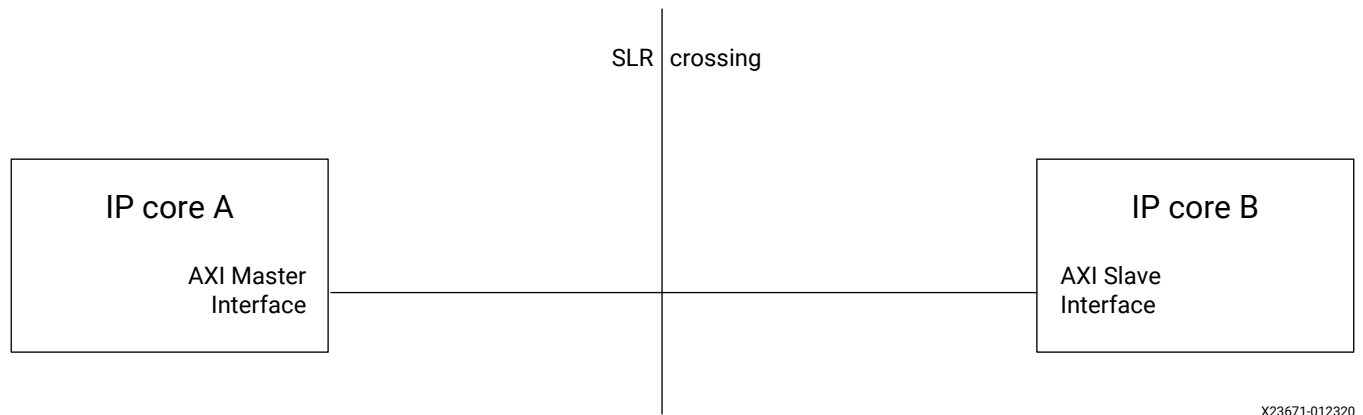
For details about synthesis and implementation, see the *Vivado Design Suite User Guide: Designing with IP* ([UG896](#)).

Timing Closure of AXI Memory-Mapped Connections Across SLRs in SSI Devices

This section describes how to apply the AXI Register Slice IP to pipeline the AXI pathways crossing between two SLR regions when targeting an SSI FPGA.

Assume two IP cores (A and B) with an AXI Memory-Mapped point-to-point connection that is known to cross from one SLR to another:

Figure 18: AXI Memory-Mapped Connections Across SLRs in SSI Devices

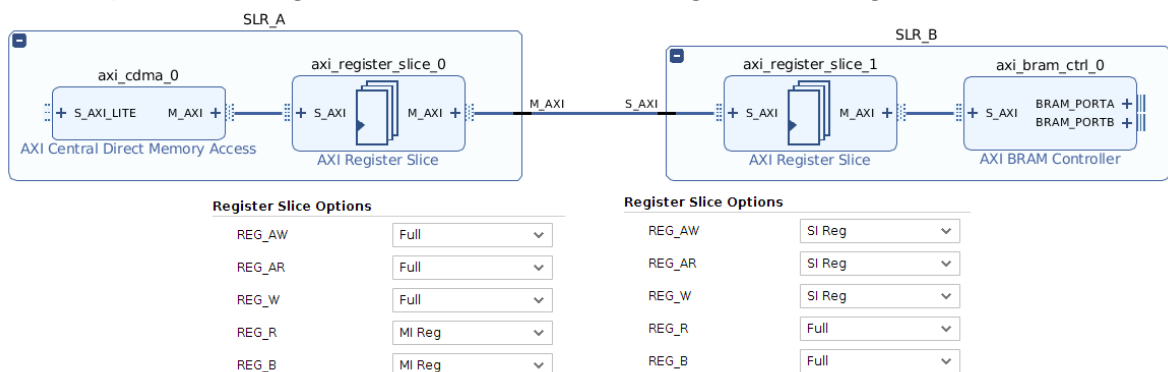


X23671-012320

To facilitate timing closure of those AXI Memory-Mapped interfaces, crossing the SLRs with flop-to-flop paths is helpful. This can be accomplished, in one of two methods, by using the Vivado IP integrator design entry.

Alternative 1: Using an Instance of AXI Register Slice in Each SLR

1. Insert two instances of AXI Register Slice into the AXI pathway connecting the master and slave endpoint IP. Configure the five channels according to the settings shown:



2. Add an XDC constraint to the design which places the master endpoint IP (core A) and the first AXI Register Slice instance to which it connects into the same SLR.

3. Add an XDC constraint to the design which places the slave endpoint IP (core B) and the second AXI Register Slice instance to which it connects into the same SLR.

See the Modes of Operation section for a description of the register options of the AXI Register Slice IP.

Related Information

[Modes of Operation](#)

Alternative 2: Using a Single Instance of AXI Register Slice to Span Multiple SLRs

1. Add a single instance of AXI Register Slice IP core into the AXI interface connection of IP cores A and B.
2. Double-click the AXI Register Slice core and configure all Register Slice Options to one of the following alternative modes, as best suited to your application:
 - **SLR TDM Crossing:** Pipelines only the crossing pathways between two adjacent SLRs using half the number of cross-SLR wires, but requiring a double-frequency clock input.
 - **Multi-SLR Crossing:** Pipelines a pathway that crosses zero or more SLRs, and can further add pipeline stages to span the distance between the SLR boundaries and the connected endpoint IP. Optionally enable **Auto-Pipeline Insertion** mode by checking the Use timing-driven pipeline insertion box.
3. Add XDC constraints to place each of the master endpoint IP (core A) and slave endpoint IP (core B) into their respective SLRs. Optionally add XDC constraint to the design which places the submodules within the AXI Register Slice core into the same SLRs as the connected endpoint IP. Constraining the AXI Register Slice submodules might be necessary if your implementation results lead to SLR crossings anywhere other than the intended pipelined pathway within the AXI Register Slice. This will typically result in timing-critical paths. See Constraining the Core sections for more details.

Related Information

[Constraining the Core](#)

Example Design

The AXI Register Slice core can generate an example design demonstrating its basic functionality when connected to a simple AXI master (traffic generator) and AXI slave. The example design system is customized to match the configuration settings you apply to the IP core. A test bench is provided to simulate the example design. The example design can also be implemented and analyzed using Vivado® Design Suite debug feature.

Upgrading

This appendix is not applicable for the first release of the core.

Debugging

This appendix includes details about resources available on the Xilinx® Support website and debugging tools.

If the IP requires a license key, the key must be verified. The Vivado® design tools have several license checkpoints for gating licensed IP through the flow. If the license check succeeds, the IP can continue generation. Otherwise, generation halts with an error. License checkpoints are enforced by the following tools:

- Vivado Synthesis
- Vivado Implementation
- write_bitstream (Tcl command)



IMPORTANT! IP license level is ignored at checkpoints. The test confirms a valid license exists. It does not check IP license level.

Finding Help on Xilinx.com

To help in the design and debug process when using the core, the [Xilinx Support web page](#) contains key resources such as product documentation, release notes, answer records, information about known issues, and links for obtaining further product support. The [Xilinx Community Forums](#) are also available where members can learn, participate, share, and ask questions about Xilinx solutions.

Documentation

This product guide is the main document associated with the core. This guide, along with documentation related to all products that aid in the design process, can be found on the [Xilinx Support web page](#) or by using the Xilinx® Documentation Navigator. Download the Xilinx Documentation Navigator from the [Downloads page](#). For more information about this tool and the features available, open the online help after installation.

Answer Records

Answer Records include information about commonly encountered problems, helpful information on how to resolve these problems, and any known issues with a Xilinx product. Answer Records are created and maintained daily ensuring that users have access to the most accurate information available.

Answer Records for this core can be located by using the Search Support box on the main [Xilinx support web page](#). To maximize your search results, use keywords such as:

- Product name
- Tool message(s)
- Summary of the issue encountered

A filter search is available after results are returned to further target the results.

Technical Support

Xilinx provides technical support on the [Xilinx Community Forums](#) for this LogiCORE™ IP product when used as described in the product documentation. Xilinx cannot guarantee timing, functionality, or support if you do any of the following:

- Implement the solution in devices that are not defined in the documentation.
- Customize the solution beyond that allowed in the product documentation.
- Change any section of the design labeled DO NOT MODIFY.

To ask questions, navigate to the [Xilinx Community Forums](#).

Debug Tools

There are many tools available to address AXI Register Slice design issues. It is important to know which tools are useful for debugging various situations.

Vivado Design Suite Debug Feature

The Vivado® Design Suite debug feature inserts logic analyzer and virtual I/O cores directly into your design. The debug feature also allows you to set trigger conditions to capture application and integrated block port signals in hardware. Captured signals can then be analyzed. This feature in the Vivado IDE is used for logic debugging and validation of a design running in Xilinx® devices.

The Vivado logic analyzer is used to interact with the logic debug LogiCORE IP cores, including:

- ILA 2.0 (and later versions)
- VIO 2.0 (and later versions)

See the *Vivado Design Suite User Guide: Programming and Debugging* ([UG908](#)).

Additional Resources and Legal Notices

Xilinx Resources

For support resources such as Answers, Documentation, Downloads, and Forums, see [Xilinx Support](#).

Documentation Navigator and Design Hubs

Xilinx® Documentation Navigator (DocNav) provides access to Xilinx documents, videos, and support resources, which you can filter and search to find information. To open DocNav:

- From the Vivado® IDE, select **Help** → **Documentation and Tutorials**.
- On Windows, select **Start** → **All Programs** → **Xilinx Design Tools** → **DocNav**.
- At the Linux command prompt, enter `docnav`.

Xilinx Design Hubs provide links to documentation organized by design tasks and other topics, which you can use to learn key concepts and address frequently asked questions. To access the Design Hubs:

- In DocNav, click the **Design Hubs View** tab.
- On the Xilinx website, see the [Design Hubs](#) page.

Note: For more information on DocNav, see the [Documentation Navigator](#) page on the Xilinx website.

References

These documents provide supplemental material useful with this guide:

1. AXI4-Stream Infrastructure IP Suite LogiCORE IP Product Guide ([PG085](#))
2. Vivado Design Suite User Guide: Designing with IP ([UG896](#))
3. Vivado Design Suite User Guide: Getting Started ([UG910](#))
4. Vivado Design Suite User Guide: Designing IP Subsystems using IP Integrator ([UG994](#))
5. Vivado Design Suite User Guide: Logic Simulation ([UG900](#))
6. Vivado Design Suite User Guide: Programming and Debugging ([UG908](#))
7. Vivado Design Suite: AXI Reference Guide ([UG1037](#))
8. UltraFast Design Methodology Guide for FPGAs and SOCs ([UG949](#))

Revision History

The following table shows the revision history for this document.

Section	Revision Summary
11/02/2022 Version 2.1	
IP Facts	Added Versal ACAP to list of supported devices.
06/30/2022 Version 2.1	
Constraining the Core	Clarified statement regarding using constraints.
12/04/2020 Version 2.1	
Initial release.	N/A

Please Read: Important Legal Notices

The information disclosed to you hereunder (the "Materials") is provided solely for the selection and use of Xilinx products. To the maximum extent permitted by applicable law: (1) Materials are made available "AS IS" and with all faults, Xilinx hereby DISCLAIMS ALL WARRANTIES AND CONDITIONS, EXPRESS, IMPLIED, OR STATUTORY, INCLUDING BUT NOT LIMITED TO WARRANTIES OF MERCHANTABILITY, NON-INFRINGEMENT, OR FITNESS FOR ANY PARTICULAR PURPOSE; and (2) Xilinx shall not be liable (whether in contract or tort, including negligence, or under any other theory of liability) for any loss or damage of any kind or nature related to, arising under, or in connection with, the Materials (including your use of the Materials), including for any direct, indirect, special, incidental, or consequential loss or damage (including loss of data, profits, goodwill, or any type of loss or damage suffered as a result of any action brought by a third party) even if such damage or loss was reasonably foreseeable or Xilinx had been advised of the possibility of the same. Xilinx assumes no obligation to correct any errors contained in the Materials or to notify you of updates to the Materials or to product

specifications. You may not reproduce, modify, distribute, or publicly display the Materials without prior written consent. Certain products are subject to the terms and conditions of Xilinx's limited warranty, please refer to Xilinx's Terms of Sale which can be viewed at <https://www.xilinx.com/legal.htm#tos>; IP cores may be subject to warranty and support terms contained in a license issued to you by Xilinx. Xilinx products are not designed or intended to be fail-safe or for use in any application requiring fail-safe performance; you assume sole risk and liability for use of Xilinx products in such critical applications, please refer to Xilinx's Terms of Sale which can be viewed at <https://www.xilinx.com/legal.htm#tos>.

AUTOMOTIVE APPLICATIONS DISCLAIMER

AUTOMOTIVE PRODUCTS (IDENTIFIED AS "XA" IN THE PART NUMBER) ARE NOT WARRANTED FOR USE IN THE DEPLOYMENT OF AIRBAGS OR FOR USE IN APPLICATIONS THAT AFFECT CONTROL OF A VEHICLE ("SAFETY APPLICATION") UNLESS THERE IS A SAFETY CONCEPT OR REDUNDANCY FEATURE CONSISTENT WITH THE ISO 26262 AUTOMOTIVE SAFETY STANDARD ("SAFETY DESIGN"). CUSTOMER SHALL, PRIOR TO USING OR DISTRIBUTING ANY SYSTEMS THAT INCORPORATE PRODUCTS, THOROUGHLY TEST SUCH SYSTEMS FOR SAFETY PURPOSES. USE OF PRODUCTS IN A SAFETY APPLICATION WITHOUT A SAFETY DESIGN IS FULLY AT THE RISK OF CUSTOMER, SUBJECT ONLY TO APPLICABLE LAWS AND REGULATIONS GOVERNING LIMITATIONS ON PRODUCT LIABILITY.

Copyright

© Copyright 2020-2022 Advanced Micro Devices, Inc. Xilinx, the Xilinx logo, Alveo, Artix, Kintex, Kria, Spartan, Versal, Vitis, Virtex, Vivado, Zynq, and other designated brands included herein are trademarks of Xilinx in the United States and other countries. All other trademarks are the property of their respective owners.